

챕터 4. 문서를 지식으로 바꾸다. 파싱, 청킹, 임베딩, ChromaDB

이번 챕터가 끝나면

- PDF, DOCX, XLSX를 **Markdown**으로 변환하는 통합 파서를 이해합니다 (Parsing)
- 고정 크기 **500자 + 100자 오버랩** 청킹으로 문맥을 보존한 조각을 만듭니다 (Chunking)
- **ko-sroberta-multitask** 임베딩으로 텍스트를 768차원 벡터로 바꾸고 **ChromaDB**에 저장합니다
- CLI 검색으로 "휴가 규정" 같은 쿼리가 실제로 원하는 문서를 찾아오는 걸 눈으로 확인합니다

준비하기. 실습 시작 전 한 번만 설정

1. 실습 폴더 이동

터미널 폴더 이동

```
cd rag-start/ex04
```

파일 구조는 다음과 같습니다.

ex04 디렉토리

```
ex04/
├─ requirements.txt
├─ data/
│   ├─ docs/                # 사내 문서 원본 (챕터 3 분류 구조)
│   ├─ markdown/           # 파싱 결과 (자동 생성)
│   └─ chroma_db/          # 벡터 DB 영속 저장소 (자동 생성)
└─ src/
    ├─ main.py              # [실습] 파이프라인 진입점 (step1, step2 분기)
    ├─ chunker.py           # [실습] Fixed-size 청킹 (500자, 100자 오버랩)
    ├─ store.py             # [실습] ko-sroberta 임베딩, ChromaDB 저장과 검색
    └─ cli_search.py        # [실습] 벡터 검색 CLI
```

```
└─ _chunk_utils.py      # [참고] chunker 내부 헬퍼
└─ _store_utils.py      # [참고] store 내부 헬퍼
└─ _search_utils.py     # [참고] cli_search 내부 헬퍼
```

2. 실습 환경 구축

터미널 환경 구성. macOS / Linux

```
cd ex04
cp .env.example .env
python3.12 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

터미널 환경 구성. Windows

```
cd ex04
copy .env.example .env
py -3.12 -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
```

임베딩 모델은 첫 실행 시 약 400MB 다운로드

`ko-sroberta-multitask` 는 최초 실행 시 **허깅페이스(HuggingFace)** 에서 자동 다운로드됩니다. 허깅페이스는 사전 학습된 AI 모델을 공개·다운로드할 수 있는 플랫폼인데, 이 책에서는 한국어 임베딩 모델을 받아 오는 출처로만 씁니다. 이후부터는 로컬 캐시를 사용하므로 한 번만 다운받으면 됩니다.

3. 사용할 라이브러리

패키지	역할
<code>pypdf</code>	PDF 텍스트 추출 (페이지별)
<code>python-docx</code>	DOCX 단락·표 추출
<code>openpyxl</code>	XLSX 셀 데이터 추출
<code>sentence-transformers</code>	ko-sroberta 임베딩 모델 로딩·실행

chromadb

벡터 DB (로컬 영속 저장소)

4. 실습 순서

1. `python src/main.py --step 1`. 파싱 결과만 먼저 확인 (Markdown)
2. `data/markdown/` 폴더 열어 변환 결과 눈으로 검증
3. `python src/main.py`. 전체 파이프라인 (파싱 → 청킹 → 임베딩 → 저장)
4. `python src/cli_search.py --query "휴가 규정"`. 검색 동작 확인

4.1 "정리는 했는데, 어떻게 넣지?"



그림 4-1. 문서를 지식으로 바꾸는 주방. 손질, 다지기, 양념, 냉장고

화요일 오전 10시. 사무실 창으로 햇살이 길게 들어오고, 모니터에는 어제 정리한 폴더 트리가 그대로 떠 있었습니다. 키보드에 얹은 손끝에 가벼운 온기가 돌았습니다.

`data/docs/hr/HR_취업규칙_v1.0.pdf` ... 6개 파일명이 단정하게 줄 맞춰져 있습니다. 옆자리 동료가 의자를 드르륵 돌렸습니다. 어제 인사팀에 한 번 혼나고 온 그 동료입니다.

동료 : "이제 진짜 잘 나오는 거죠?"

잘 나와야지.

챗터 3에서 머릿속에 그려둔 규칙(Markdown 통일, 폴더 메타데이터, 500자에 100자 오버랩)을 이제 **코드로 옮길 차례**입니다. 손질하고, 다지고, 양념하고, 냉장고에 정리하면 6개 문서가 검색 가능한 지식이 됩니다.

4.2 네 단계 파이프라인

팀장 : "재료 손질 안 하고 요리해 본 적 있어요?"

마트에서 사 온 재료를 봉지째 냉장고에 던지면 어떻게 될까요? 나중에 찾을 수 없습니다. 양파가 어디 있는지, 고기가 아직 썰 만한지 다 뒤져야 합니다. 제대로 하려면 네 단계를 거칩니다.

1. **손질**. 흙 씻고, 껍질 벗기고, 뼈 발라냄. 먹을 수 없는 부분 제거.
2. **다지기**. 요리에 맞게 적당한 크기로 자름.
3. **양념**. 소금에 절이거나 밑간. 나중에 바로 쓸 수 있게.
4. **냉장고 정리**. 라벨 붙이고 구분해서 넣음.

사내 문서도 같습니다.

주방	문서 처리	무슨 일
손질	파싱 (Parsing)	PDF·DOCX·XLSX에서 텍스트를 꺼내 Markdown으로 통일
다지기	청킹 (Chunking)	Markdown을 500자 조각으로 자름 (100자 오버랩)
양념	임베딩 (Embedding)	조각을 768차원 숫자 벡터로 변환
냉장고	벡터 DB 저장	벡터를 ChromaDB에 넣어 유사도 검색 가능하게

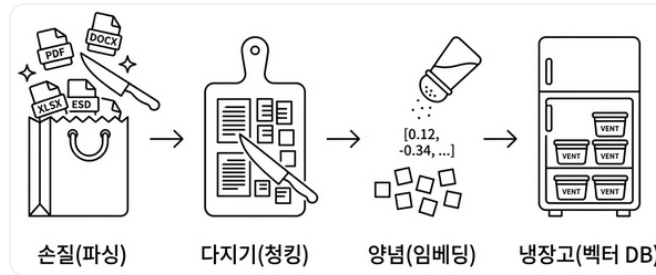


그림 4-2. 손질 → 다지기 → 양념 → 냉장고 정리. 이 네 단계가 오프라인 파이프라인의 본체입니다

4.3 손질 - 파싱부터 돌려본다

호기심에 `HR_취업규칙_v1.0.pdf` 를 파이썬으로 그냥 `open()` 해봤습니다. 화면에 쏟아진 건 `%PDF-1.6\n%âãÏÓ...` 로 시작하는 뜻 모를 문자열이었습니다.

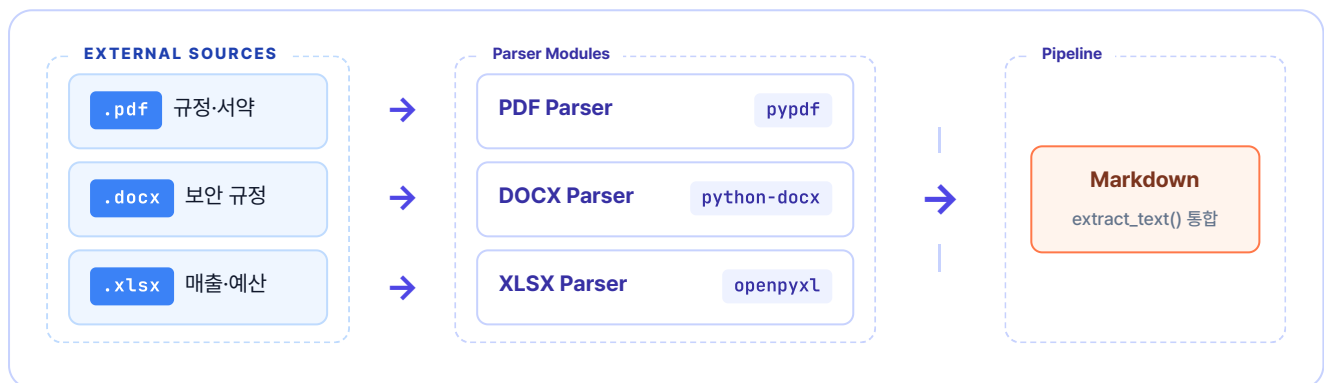
아, 그냥 열면 안 되는구나.



그림 4-3. 취업규칙 PDF 원본. 사람 눈에는 글자지만 컴퓨터엔 바이너리입니다

첫 단계는 **손질**입니다. 사람 눈에는 "제1조 제1항"이 보이지만 컴퓨터엔 PDF 파일이 그냥 바이너리입니다. 텍스트를 꺼내려면 **파서(Parser)** 가 필요합니다. 형식마다 쓰는 도구가 다릅니다.

형식	파서	특징
PDF	<code>pypdf</code>	텍스트 기반 PDF에서 페이지별 추출. 이미지 PDF는 챗터 10에서 OCR·Vision으로 처리
DOCX	<code>python-docx</code>	단락(Paragraph)과 표(Table) 추출, 제목을 Markdown으로 변환
XLSX	<code>openpyxl</code>	시트별 셀 데이터를 행 단위로 읽음



전략 패턴(Strategy). 호출자는 `extract_text(path)` 하나만 알면 되고, 새 형식이 늘어도 매핑 한 줄만 추가하면 됩니다.

그림 4-4. 파일이 들어오면 확장자에 따라 파서가 선택되고, 통합 텍스트(Markdown)로 출력됩니다

`ex04/src/extractor.py`의 통합 함수가 확장자를 보고 파서를 자동 선택합니다. 이 함수는 이미 구현된 상태이고 우리는 `ex04/src/main.py`에서 호출만 합니다.

extractor의 전략 패턴. `ex04/src/extractor.py`의 `extract_text()` 함수는 `{"pdf": extract_from_pdf, "docx": extract_from_docx, "xlsx": extract_from_xlsx}` 같은 딕셔너리 매핑으로 파서를 고릅니다. 새 형식을 추가할 때 딕셔너리에 한 줄만 추가하면 되는 구조입니다.

욕심 같아선 `python src/main.py` 한 번에 전체 파이프라인을 돌리고 싶지만, **파싱 결과부터 따로 확인**하는 게 안전합니다. 챗터 3에서 본 **Garbage In, Garbage Out** 원칙이 여기서 적용됩니다. 입력이 깨지면 뒤에서 아무리 청킹과 임베딩을 잘해도 복구되지 않습니다.

그래서 `ex04/src/main.py`는 4단계를 두 **Step**으로 묶어 쪼개 뒀습니다.

단계	함수	하는 일	실행 옵션
Step 1	<code>step1_python_parsing()</code>	소스질(파싱) → 결과를 <code>data/markdown/</code> 에 저장	<code>--step 1</code>
Step 2	<code>step2_embed_and_store()</code>	단어(청킹) → 양념 (임베딩) → 냉장고 (ChromaDB 저장)	<code>--step 2</code> (없으면 1, 2 모두)

Step 1을 먼저 돌려 `data/markdown/` 결과물을 눈으로 검증하고, 괜찮으면 Step 2로 넘어갑니다. `-step` 인자를 빼면 전체가 순서대로 실행됩니다. 이 섹션(4.3)에서는 Step 1을, 다음 섹션(4.4)에서 Step 2를 채웁니다.

`ex04/src/main.py`를 열고 `step1_python_parsing` 함수의 TODO 주석 아래 `pass`를 지우고 아래 코드를 작성합니다.

실습 1 ex04/src/main.py. `step1_python_parsing`

```
def step1_python_parsing(docs_dir: str) → list[dict]:
    # TODO: extract_all_from_directory()로 문서 추출 → 결과 출력 → 마크다운 저장
    # 1. docs_dir(data/docs/) 안의 PDF/DOCX/XLSX를 한꺼번에 파싱
    results = extract_all_from_directory(docs_dir)
    # 2. 파싱 결과를 data/markdown/에 .md 파일로 저장 (눈으로 확인용)
    save_results_as_markdown(results)
    return results
```

`extract_all_from_directory()`는 `ex04/src/extractor.py`에 이미 구현돼 있고 `save_results_as_markdown()`은 `_pipeline_utils.py`의 헬퍼입니다. 우리가 하는 일은 둘을 순서대로 호출하는 것뿐입니다.

이어서 같은 파일 `ex04/src/main.py`의 `main()` 함수 분기 TODO의 `pass`를 지우고 아래 코드를 작성합니다. `step1_python_parsing`을 `--step 1` 인자에 연결합니다.

실습 1 ex04/src/main.py. `main()` 분기

```
def main() → None:
    args = parse_arguments()
```

```
steps_to_run = sorted(set(args.step))
python_results: list[dict] = []

# TODO: 1 in steps_to_run이면 step1_python_parsing 실행
if 1 in steps_to_run:
    python_results = step1_python_parsing(docs_dir=args.docs_dir)
```

이제 파싱만 실행합니다.

터미널 파싱만 실행

```
python src/main.py --step 1
```

`--step 1` 은 파싱과 Markdown 변환만 수행합니다. 끝나면 `data/markdown/` 에 변환 결과가 생깁니다. `HR_취업규칙_v1.0.md` 를 열어 확인합니다.

```
data/markdown/HR_취업규칙_v1.0.md
```

```
# HR_취업규칙_v1.0.pdf
```

- 파일 형식: pdf
- 추출 글자 수: 1916자

```
---
```

```
## 페이지 1
```

```
취업규칙 (다단 편집형) 문서번호: HR-2026-001
버전: v2.0 (Draft)
```

```
...
```

파일 정보가 상단에 요약돼 있고 `## 페이지 1` 단위로 텍스트가 잘 추출됐습니다.

이번엔 XLSX 결과를 확인합니다. `data/markdown/FIN_부서별_예산기안서.md` 를 열면 시트가 **Markdown 표 형식**(`| 열1 | 열2 |`)으로 변환돼 있습니다.

```
data/markdown/FIN_부서별_예산기안서.md
```

```
# FIN_부서별_예산기안서.xlsx
```

- 파일 형식: xlsx

[시트: 예산기안서_최종]

2025년도 주요사업부 예산 집행 기안서									
---	---	---	---	---	---				
문서번호:	FIN-BDG-2025-001	결재	기안	검토	승인				
기안부서:	재무전략실								
#	본부명	부서명	예산	계정	Q1	배정액(원)	Q2	배정액(원)	
1	경영지원본부	인사총무팀	복리후생비		25000000		30000000		

...

표 모양이 살짝 삐뚤해 보여도 벡터 DB 입장에서는 **원본 엑셀보다 훨씬 이해하기 쉬운** 형태입니다. 헤더 행과 구분선(---)이 있고 파이프(|) 기호로 행, 열 관계가 텍스트 흐름에 담겨 있어, "경영지원본부 예산"을 검색하면 표의 해당 데이터가 정확히 잡힙니다.

4.4 다지기, 양념, 냉장고, 전체 파이프라인

파싱이 깨끗한 걸 확인했으면 나머지 세 단계를 한 번에 돌립니다.

4.4.1 다지기 - 청킹

취업규칙 전문을 한 덩어리로 넣으면 챕터 1에서 겪은 바로 그 문제가 반복됩니다. 관련 없는 내용까지 떨어웁니다. 챕터 3에서 설계한 대로 **고정 크기 500자 + 100자 오버랩**으로 자릅니다. 청크가 어떻게 잘리고 어떻게 겹치는지는 챕터 3.6 오버랩 도식에서 이미 눈으로 봤으니, 여기서는 구현만 짚습니다.

ex04/src/chunker.py 를 열고 split_text_into_chunks() 함수 안 TODO의 pass 를 지우고 아래 코드를 작성합니다.

실습 2 ex04/src/chunker.py. split_text_into_chunks

```
def split_text_into_chunks(
    text: str,
    chunk_size: int = DEFAULT_CHUNK_SIZE,
    overlap: int = DEFAULT_OVERLAP,
) → list[str]:
    if chunk_size ≤ overlap:
        raise ValueError(
            f"chunk_size({chunk_size})는 overlap({overlap})보다 커야 합니다."
        )
```

```

text = text.strip()
if not text:
    return []

# TODO: chunk_size 단위로 텍스트를 자르되, overlap만큼 겹치게 합니다
chunks = [] # 결과를 담은 리스트
step = chunk_size - overlap # 다음 청크 시작 위치 (500-100=400자씩 이동)
start = 0 # 현재 위치

while start < len(text): # 텍스트 끝까지 반복
    end = start + chunk_size # 현재 위치에서 500자 잘라내기
    chunk = text[start:end].strip() # 앞뒤 공백 제거
    if chunk: # 빈 문자열이 아니면
        chunks.append(chunk) # 결과에 추가
        start += step # 400자 뒤로 이동 (100자 겹침)

return chunks

```

`DEFAULT_CHUNK_SIZE` (500), `DEFAULT_OVERLAP` (100)은 `chunker.py` 상단에 선언된 모듈 상수입니다. `main.py` 에서 `--chunk-size` 로 넘겨 덮어쓸 수 있습니다.

`step = chunk_size - overlap` 이 핵심입니다. 500자를 자르지만 다음 청크는 400자 뒤에서 시작하므로 결과적으로 **100자가 겹칩니다**. `while` 루프가 텍스트 끝까지 반복하며 청크 리스트를 쌓아 올립니다.

왜 overlap이 필요한가. 한 청크의 끝 문장이 다음 청크에 일부 포함되어야 의미가 잘리지 않습니다. 만약 overlap이 0이면 문장이 청크 경계에서 정확히 끊겨, 검색 결과로 받은 청크에 "...연차는 입사 1년 후" 같이 어중간한 문장이 들어올 수 있습니다. 100자 정도를 겹치게 두면 그런 경계 손실을 막아 줍니다. 너무 크게 하면 중복이 늘어 비용이 커지므로 보통 `chunk_size`의 **15~25%** 가 권장 범위입니다.

각 조각에는 **메타데이터**도 함께 붙입니다. 출처 파일명, 페이지, 분류. 챕터 3에서 설계한 라벨이 여기서 쓰이고, 나중에 에이전트가 "이 답변의 출처는 취업규칙 3페이지입니다"라고 당당하게 말할 수 있게 됩니다.

청크 한 조각 = 본문 + 메타데이터 라벨

제5조(연차유급휴가) 신입사원은 입사 후 3년 동안은 연차가 없다. 대신 매월 1회 '리프레시 데이'를 유급으로 제공한다. 3년 근속 시점에 30일의 연차가 일시에 발생한다. ...

출처 HR_취업규칙_v1.0.pdf

페이지 3

분류 HR

chunk_id hr_취업규칙_v1_0_text_p003_c0002

이 라벨들이 ChromaDB에 `metadata` 필드로 같이 저장됩니다. 검색할 때 "HR 폴더의 3페이지 근처 문서만"처럼 필터링도 가능하고, 답변에 출처를 붙일 때도 이 값을 그대로 씁니다.

4.4.2 양념 - 임베딩

컴퓨터는 글자를 모릅니다. 숫자만 연산합니다. **임베딩(Embedding)**은 텍스트의 의미를 숫자 벡터(768개 숫자 리스트)로 바꾸는 과정입니다. 핵심은 **의미가 비슷한 텍스트는 비슷한 숫자**가 된다는 점입니다.

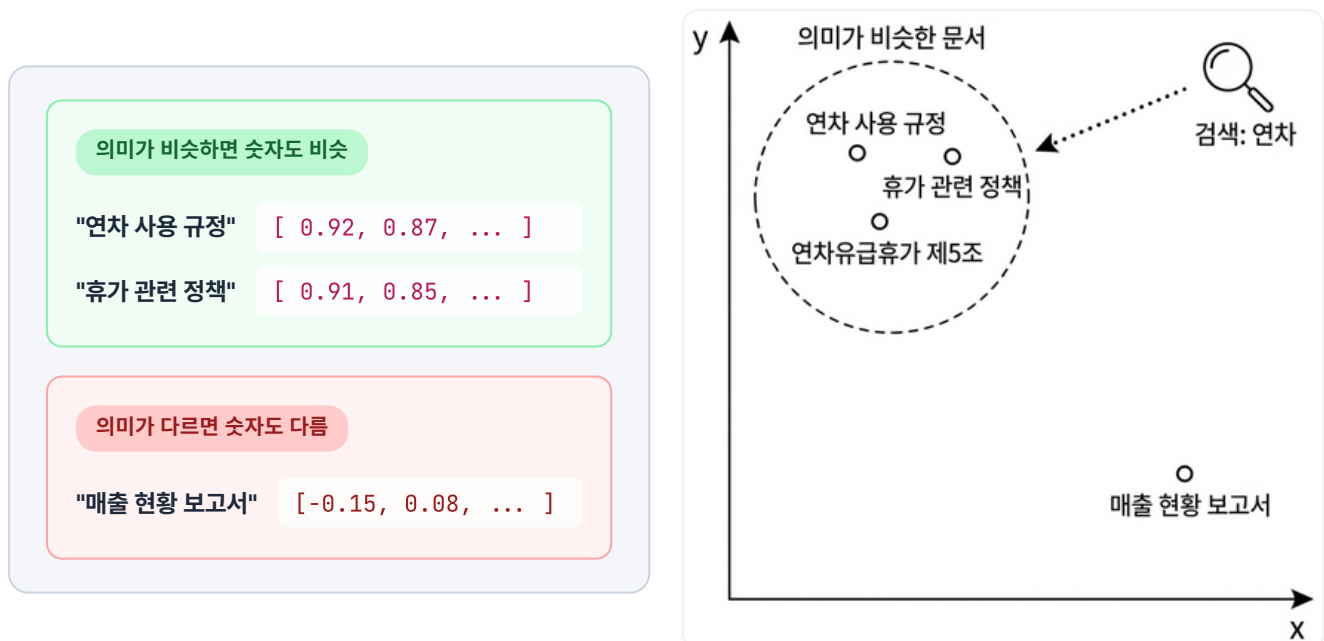


그림 4-5. 왼쪽(문장 임베딩 벡터 예시. 앞 몇 자리만 봐도 비슷/다름이 드러남), 오른쪽(임베딩 공간에 찍힌 좌표. 의미가 가까운 문서가 한 곳에 모임)

"숫자가 비슷하다"는 걸 어떻게 판단할까요? 벡터를 좌표 위의 **화살표**라고 생각하시면 됩니다. 두 화살표가 같은 방향이면 의미가 비슷하고, 반대면 다릅니다. **코사인 유사도(Cosine Similarity)**가 이 각도를 측정합니다.

- 같은 방향(각도 0°) → 유사도 **1** (완전히 같은 의미)

- 직각(90°) → 유사도 **0** (관련 없음)
- 반대 방향(180°) → 유사도 **-1** (반대 의미)

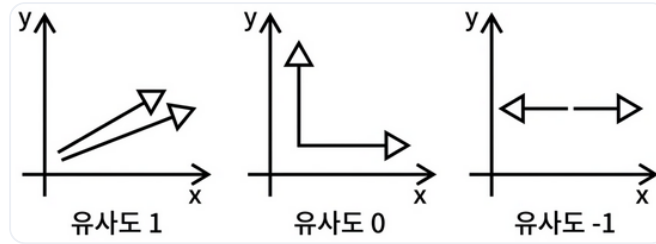


그림 4-6. 코사인 유사도. 각도로 의미 유사도를 측정합니다

왜 코사인 유사도? — 유클리드 거리와의 차이. 두 벡터의 가까움을 재는 방법은 두 가지가 있습니다. **유클리드 거리**는 두 점의 직선 거리를 재고, **코사인 유사도**는 두 화살표의 각도를 잽니다. 텍스트 임베딩에서는 같은 의미라도 문장 길이에 따라 벡터 크기(magnitude)가 달라질 수 있어, 길이를 무시하고 방향만 보는 코사인 유사도가 더 안정적입니다. ChromaDB의 기본 metric도 코사인 유사도입니다.

임베딩 모델은 여럿 있지만 이 책은 **ko-sroberta-multitask**를 씁니다. 한국어 문장 유사도에 파인튜닝돼 있고("연차"와 "휴가"가 가깝다는 걸 잘 잡습니다), 로컬 실행이라 API 비용이 없고, 사내 민감 정보를 외부로 보내지 않아도 됩니다.

챕터 8에서 다른 모델과 비교

챕터 8에서 ko-sroberta와 다른 임베딩 모델의 검색 품질을 비교 실험합니다. 지금은 기본값으로 시작하고, 더 나은 선택지가 있는지는 챕터 8에서 확인합니다.

4.4.3 냉장고 - ChromaDB 저장

양념까지 끝난 재료를 **ChromaDB**에 정리합니다. 챕터 1에서 `Chroma.from_documents()` 한 줄로 썼지만, 이번에는 `ex04/src/store.py` 가 직접 넣습니다.

ChromaDB란? 텍스트 의미를 담은 **벡터**(768차원 숫자 배열)를 저장하고, 주어진 질문 벡터와 가장 가까운 벡터를 빠르게 찾아 주는 **벡터 데이터베이스**입니다. 일반 관계형 DB(PostgreSQL, MySQL)가 "이름이 홍길동인 행"처럼 정확한 값을 조회한다면, 벡터 DB는 "이 질문과 의미가 비슷한 문서 Top-K"를 조회합니다. 로컬에 파일로 영속 저장이 가능해 서버 재시작 후에도 데이터가 남아 있고, Python 한 줄로 띄울 수 있을 정도로 가볍습니다. 실무에서는 Pinecone, Weaviate, Milvus 같은 유료/대규모용 옵션도 쓰지만, 이 책처럼 로컬 실습, 소규모 운영에는 ChromaDB가 표준에 가깝습니다.

저장 항목	내용	예시
id	청크 고유 ID	hr_취업규칙 _v1_0_text_p001_c0003
document	청크 텍스트 원문	"제5조 연차유급휴가는..."
embedding	768차원 벡터	[0.12, -0.34, ...]
metadata	출처 정보	{file_name: "HR_취업규칙 _v1.0.pdf", page: 3}

저장은 **업서트(upsert)**. 같은 ID가 있으면 덮어쓰고 없으면 새로 추가합니다. 파이프라인을 여러 번 실행해도 중복이 쌓이지 않습니다. 챗터 3에서 설계한 **전체 재인덱싱** 전략과 정확히 맞닿습니다.

ex04/src/store.py 에는 _store_utils.py 에서 import해 쓰는 **모듈 상수 3개**가 함수 시그니처 기본값으로 등장합니다. CLI 옵션으로 덮어쓸 수 있습니다.

상수	기본값	역할
DEFAULT_CHROMA_DIR	./data/chroma_db	ChromaDB가 벡터를 저장할 로컬 디렉토리. 서버가 재시작돼도 데이터 유지
DEFAULT_COLLECTION_NAME	metacoding_documents	컬렉션 이름(일반 DB의 "테이블" 같은 개념). 프로젝트별로 다르게 쓰면 여러 벡터DB를 한 디렉토리에 공존 가능

DEFAULT_EMBEDDING_MODEL

jhgan/ko-sroberta-
multitask

텍스트를 768차원 벡터로 변환할
임베딩 모델(HuggingFace ID).
검색 시에도 같은 모델을 써야
의미가 맞음

ex04/src/store.py 를 열고 store_chunks_to_chroma() 함수의 TODO의 pass 를 지우고
아래 코드를 작성합니다. 4단계를 순서대로 구현합니다.

실습 3 ex04/src/store.py. store_chunks_to_chroma

```
def store_chunks_to_chroma(
    chunks: list[dict],
    chroma_dir: str = DEFAULT_CHROMA_DIR,
    collection_name: str = DEFAULT_COLLECTION_NAME,
    embedding_model_name: str = DEFAULT_EMBEDDING_MODEL,
) → dict:
    if not chunks:
        raise ValueError("저장할 청크가 없습니다.")
    chroma_dir = str(Path(chroma_dir).resolve())

    # TODO: 위 4단계를 순서대로 구현합니다

    # 1. ko-sroberta 임베딩 모델 로드 (최초 실행 시 ~400MB 다운로드, 이후 캐시)
    model = load_embedding_model(embedding_model_name)

    # 2. ChromaDB 클라이언트 생성 - 디스크에 영속 저장 (프로그램 종료해도 유지)
    client = chromadb.PersistentClient(
        path=chroma_dir,
        settings=Settings(anonymized_telemetry=False),
    )
    # 컬렉션이 없으면 생성, 있으면 기존 컬렉션 반환. cosine 유사도 사용
    collection = get_or_create_collection(client, collection_name)

    # 3. 청크 텍스트를 768차원 벡터로 변환 + ChromaDB용 데이터 정리
    ids, documents, embeddings, metadatas = embed_chunks(chunks, model)

    # 4. 배치 단위로 upsert - 같은 ID가 있으면 덮어쓰기 (중복 방지)
    for batch_start in range(0, len(ids), BATCH_SIZE):
        batch_end = batch_start + BATCH_SIZE
        collection.upsert(
            ids=ids[batch_start:batch_end],
            documents=documents[batch_start:batch_end],
            embeddings=embeddings[batch_start:batch_end],
            metadatas=metadatas[batch_start:batch_end],
        )
```

```

return {
    "collection_name": collection_name,
    "chroma_dir": chroma_dir,
    "total_chunks": len(chunks),
    "collection_count": collection.count(),
}

```

`_store_utils.py`의 헬퍼 함수 세 개가 코드에 쓰였습니다. 직접 구현하진 않지만 역할은 알아 두세요.

함수	역할
<code>load_embedding_model(name)</code>	HuggingFace에서 <code>ko-sroberta-multitask</code> 모델을 로드해 <code>SentenceTransformer</code> 객체를 반환. 최초 1회만 다운로드, 이후 캐시
<code>get_or_create_collection(client, name)</code>	ChromaDB 컬렉션을 가져오거나 없으면 생성. 유사도 계산 방식을 cosine 으로 지정
<code>embed_chunks(chunks, model)</code>	청크 리스트를 (<code>ids</code> , <code>documents</code> , <code>embeddings</code> , <code>metadatas</code>) 네 리스트로 분해. 문서 텍스트를 배치 단위로 768차원 벡터로 변환

우리가 할 일은 이 세 함수를 **순서대로 호출**하는 것뿐입니다.

2단계에서 `chromadb.PersistentClient(path=chroma_dir)`가 **벡터DB를 여는** 부분입니다. 챕터 1에서는 `Chroma.from_documents()` 한 줄이 메모리에 임시 저장소를 만들고 프로그램이 끝나면 사라졌습니다. 사내 실서비스에선 매번 문서를 다시 임베딩하는 건 낭비라서, 이번엔 **디스크에 파일로 영속 저장**합니다. `path`에 적은 폴더(`data/chroma_db/`)에 벡터가 파일로 쌓이고, 프로그램을 켜다 켜도 그대로 남아 있습니다. `Settings(anonymized_telemetry=False)`는 ChromaDB가 기본으로 켜 둔 익명 사용 통계 수집을 끄는 옵션. 사내 문서를 다루는 실습에선 꺼 두는 편이 안전합니다.

`chromadb.Client` vs `chromadb.PersistentClient`. 같은 ChromaDB지만 여는 방식이 다릅니다. `Client()`는 프로세스 메모리에만 존재하는 임시 저장소(프로그램 종료 시 휘발)고, `PersistentClient(path=...)`는 지정 디렉토리에 SQLite + Parquet 파일로 저장해 **재시작 후에도 유지**됩니다. 대규모 운영에선 Pinecone, Weaviate처럼 서버를 별도로 띄우지만, 로컬 실습이나 소규모 사내 시스템에는 `PersistentClient`만으로 충분합니다.

3단계에서 `embed_chunks()` 가 청크를 768차원 벡터로 일괄 변환하고, 4단계 `collection.upsert()` 가 이를 `BATCH_SIZE` (기본값 64)씩 끊어 저장합니다. `upsert` 는 같은 ID가 있으면 덮어쓰는 연산이라, 파이프라인을 여러 번 돌려도 중복이 쌓이지 않습니다. 메모리가 부족하면 `BATCH_SIZE` 를 32로 줄이고, 여유가 있고 임베딩이 느리면 128로 늘릴 수 있습니다.

4.4.4 실습 - main.py로 한 번에 돌리기

`ex04/src/main.py` 를 열고 `step2_embed_and_store` 함수의 TODO의 `pass` 를 지우고 아래 코드를 작성합니다. 앞에서 파싱이 돌려둔 결과(`python_results`)를 받아 청킹·임베딩·저장 두 함수를 순서대로 호출합니다.

실습 4 ex04/src/main.py. step2_embed_and_store

```
def step2_embed_and_store(
    python_results: list[dict],
    chroma_dir: str,
    collection_name: str,
    embedding_model_name: str,
    chunk_size: int = DEFAULT_CHUNK_SIZE,
    overlap: int = DEFAULT_OVERLAP,
) → dict:
    # TODO: chunk_all_documents()로 청킹 → store_chunks_to_chroma()로 저장
    # 1. 파싱 결과를 500자 + 100자 오버랩으로 청킹
    all_chunks = chunk_all_documents(python_results, chunk_size, overlap)
    # 2. 청크를 임베딩하여 ChromaDB에 저장
    store_result = store_chunks_to_chroma(
        chunks=all_chunks,
        chroma_dir=chroma_dir,
        collection_name=collection_name,
        embedding_model_name=embedding_model_name,
    )
    return store_result
```

마지막으로 같은 `ex04/src/main.py` 의 `main()` 안 `--step 2` 분기 TODO의 `pass` 를 지우고 아래 코드를 작성합니다. (`--step 2` 만 골라 돌리면 파싱이 없으므로 내부에서 step1을 다시 부릅니다.)

실습 4 ex04/src/main.py. main() 의 step2 분기

```
# TODO: 2 in steps_to_run이면 step2_embed_and_store 실행
if 2 in steps_to_run:
```



```
if not python_results:
    python_results = step1_python_parsing(docs_dir=args.docs_dir)
step2_embed_and_store(
    python_results=python_results,
    chroma_dir=args.chroma_dir,
    collection_name=args.collection,
    embedding_model_name=args.embedding_model,
    chunk_size=args.chunk_size,
    overlap=args.overlap,
)
```

이제 전체 파이프라인을 한 번에 돌립니다.

터미널 전체 파이프라인 실행

```
python src/main.py
# 청크 크기를 바꾸고 싶다면
python src/main.py --chunk-size 300 --overlap 50
```

명령 한 줄로 파싱부터 저장까지 어떤 순서로 흘러가는지 한눈에 보면 다음과 같습니다.

PYTHON SRC/MAIN.PY 실행 흐름

\$ python src/main.py

↓

main()

parse_arguments() → steps_to_run = [1, 2]

↓

1 step1_python_parsing **파싱**

extract_all_from_directory(docs_dir)

| data/docs/ 안 6개 파일(PDF·DOCX·XLSX) 파싱 → list[dict]

save_results_as_markdown(results)

| data/markdown/*.md 저장 (눈으로 확인 가능)

↓ python_results 전달

2 step2_embed_and_store **청킹·임베딩·저장**

chunk_all_documents(python_results, 500, 100)

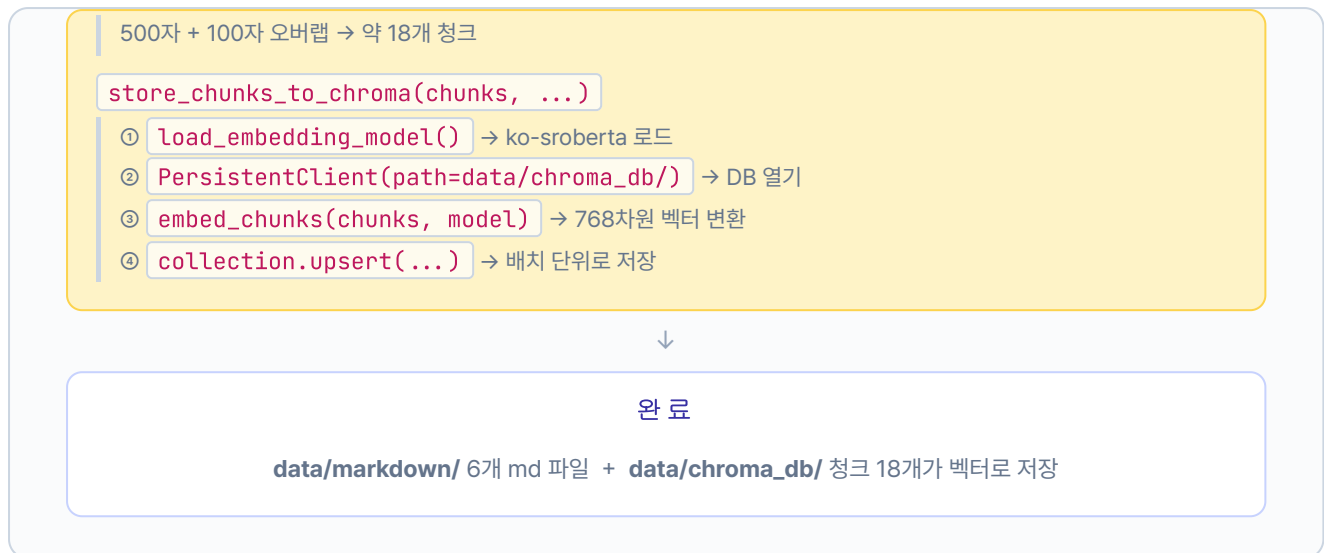


그림 4-7 (왼쪽). Step 1 — 6개 사내 문서가 Markdown으로 변환

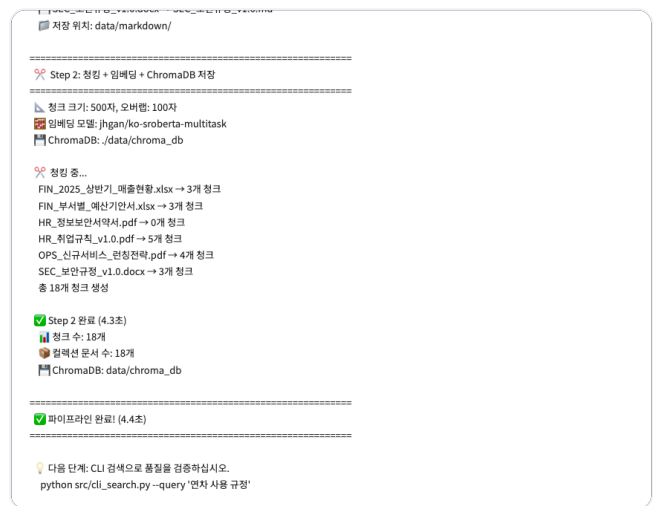


그림 4-7 (오른쪽). Step 2 — 18개 청크가 ChromaDB에 저장

4.5 검색 확인 - "휴가 규정"을 찾아오는가

손질·다지기·양념·냉장고 정리까지 끝났으니, 이제 **냉장고를 열어 재료를 꺼내는** 단계입니다. 사용자의 질문 (예: "휴가 규정")이 들어오면 같은 임베딩 모델로 질문도 벡터화한 뒤, ChromaDB에서 가장 가까운 청크를 찾아 돌려줍니다. `ex04/src/store.py` 를 열고 `search_chroma()` 함수의 TODO의 `pass` 를 지우고 아래 코드를 작성합니다.

실습 5 ex04/src/store.py. search_chroma

```
def search_chroma(
    query: str,
    chroma_dir: str = DEFAULT_CHROMA_DIR,
    collection_name: str = DEFAULT_COLLECTION_NAME,
    embedding_model_name: str = DEFAULT_EMBEDDING_MODEL,
    top_k: int = 5,
) → list[dict]:
    chroma_dir_path = Path(chroma_dir)
    if not chroma_dir_path.exists():
        print(f"ChromaDB 디렉토리가 없습니다: {chroma_dir}")
        print("main.py를 먼저 실행하여 문서를 색인하십시오.")
        sys.exit(1)

    # TODO: 쿼리 임베딩 → collection.query() → 결과 정리

    # 1. 질문 텍스트를 벡터로 변환 (저장할 때와 같은 모델 사용)
    model = load_embedding_model(embedding_model_name)
    query_embedding = model.encode([query], normalize_embeddings=True).tolist()

    # 2. 저장된 ChromaDB를 열고 컬렉션 가져오기
    client = chromadb.PersistentClient(
        path=str(chroma_dir_path.resolve()),
        settings=Settings(anonymized_telemetry=False),
    )
    collection = client.get_collection(name=collection_name)

    # 3. 쿼리 벡터와 가장 가까운 청크 top_k개 검색
    results = collection.query(
        query_embeddings=query_embedding,
        n_results=top_k,
        include=["documents", "distances", "metadatas"],
    )

    # 4. 검색 결과를 순위별 딕셔너리 리스트로 정리
    search_results = []
    docs = results.get("documents", [[]])[0]
    dists = results.get("distances", [[]])[0]
    metas = results.get("metadatas", [[]])[0]

    for rank, (doc, dist, meta) in enumerate(zip(docs, dists, metas), start=1):
        search_results.append({
            "rank": rank, "text": doc,
            "distance": round(dist, 4), "metadata": meta,
        })

    return search_results
```

저장과 똑같은 임베딩 모델을 써야 합니다. 질문 벡터와 저장된 청크 벡터가 같은 공간에서 비교되어야 거리가 의미를 갖기 때문입니다. 검색 결과는 `{rank, text, distance, metadata}` 형태의 리스트로 정리돼 돌아옵니다.

`cli_search.py` 는 `_search_utils.py` 의 `format_distance_as_similarity()` 헬퍼를 써서 ChromaDB 거리(0~2)를 백분율 유사도(0~100%)로 바꿔 출력합니다. 거리가 0에 가까울수록 유사도는 100%에 가까워집니다.

터미널 CLI 검색

```
python src/cli_search.py --query "휴가 규정" --top-k 3
```

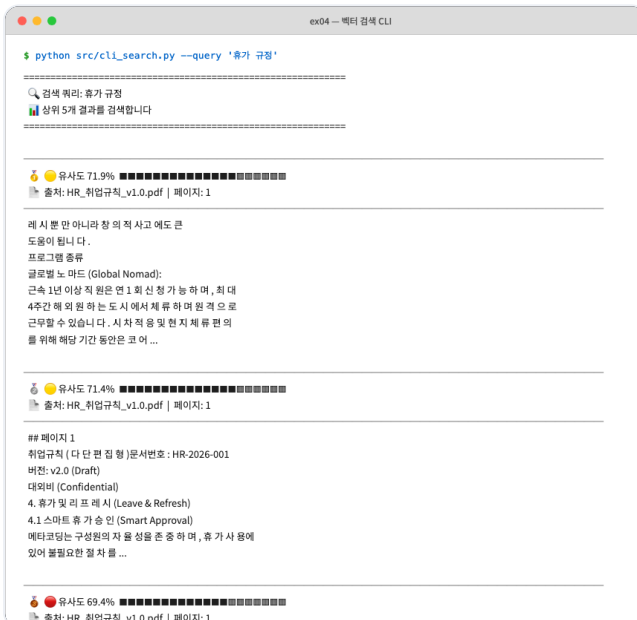


그림 4-8 (왼쪽). 쿼리와 상위 1, 2위 검색 결과



그림 4-8 (오른쪽). 3위 결과. 취업규칙의 휴가, 연차 관련 조항이 잡혔습니다

1~3위가 모두 취업규칙 휴가 관련 내용입니다. 키워드가 정확히 일치하지 않아도 의미가 가까우면 찾아냅니다.

옆자리 동료가 화면을 슬쩍 들여다봤습니다.

동료 : "이제 인사팀 안 가도 되겠네요."

웃음이 한 번 새어 나왔습니다. 그러나 화면을 다시 보니 1위 유사도가 **71%** 였습니다. `--top-k 5` 로 늘려 보니 4, 5위에 결이 다른 청크가 섞여 있었습니다.

아직 완벽하지는 않다.

벡터 DB는 "그나마 가까운 순서"로 k개를 **무조건** 채웁니다. 그래서 검색 개수를 늘리면 관련 없는 문서가 억지로 달려와요. 검색 품질을 본격적으로 끌어올리는 작업은 챕터 8에서 다룹니다. 지금은 "휴가 규정"에 취업규칙이 1위로 잡히는 것까지가 이번 챕터의 도착선입니다.

검색 결과는 PC, 모델 버전에 따라 조금 다를 수 있습니다

임베딩 모델 버전이나 실행 환경에 따라 순서, 유사도 수치가 책과 달라질 수 있습니다. 최상위(Top 1~3)에 취업규칙의 휴가, 연차 관련 내용이 잡히면 정상입니다.

4.6 더 알아보기

왜 Markdown으로 변환할까요? 임베딩 모델은 훈련 데이터에 Markdown이 많이 포함되어 있어 `# 제목`, `| 표 |`, `- 목록` 같은 구조를 잘 이해합니다. PDF 바이너리를 그대로 넣는 것보다 Markdown 텍스트가 검색 품질이 좋습니다. 그리고 `data/markdown/`의 결과물을 **사람이 눈으로 검증**할 수 있다는 이점도 있습니다.

이미지 기반 PDF의 한계. 예제 파일 중 `HR_정보보안서약서.pdf`는 스캔한 이미지로만 구성돼 있어 `pypdf`가 텍스트를 뽑지 못합니다. `extractor.py`의 PDF 파서는 이런 파일에서 빈 문자열을 돌려주고, 결국 청크가 0개로 나옵니다. 이미지 기반 PDF는 OCR이나 Vision LLM이 필요한데, 이 문제는 **챕터 10**에서 해결합니다.

extractor.py의 전략 패턴. 파일 확장자에 따라 파서를 자동으로 고르는 구조는 딕셔너리 매핑 한 줄로 끝납니다.

설명 ex04/src/extractor.py. 전략 패턴 (발췌)

```
extractor_map = {
    ".pdf": extract_from_pdf,
    ".docx": extract_from_docx,
    ".xlsx": extract_from_xlsx,
}
return extractor_map[suffix](file_path)
```

새 형식(예: HWP)을 추가하려면 이 딕셔너리에 한 줄 추가하고 해당 파서 함수만 구현하면 됩니다.

`if/elif` 로 분기하는 것보다 확장이 쉽고, 파서끼리 결합도도 낮습니다.

청킹 · 임베딩 옵션 한눈에. 자주 손보는 값들입니다.

상수	기본값	언제 바꾸나
<code>chunk_size</code>	500자	규정처럼 짧은 조항은 300자, 긴 보고서는 800자까지 실험
<code>overlap</code>	100자	<code>chunk_size</code> 의 20% 전후가 무난. 경계에서 잘려도 되는 문서면 0으로
<code>BATCH_SIZE</code>	64	메모리 부족하면 32, 여유 있으면 128
<code>top_k</code> (검색)	5	정확도만 본다면 3, 리랭커 입력 후보군 만들려면 10+

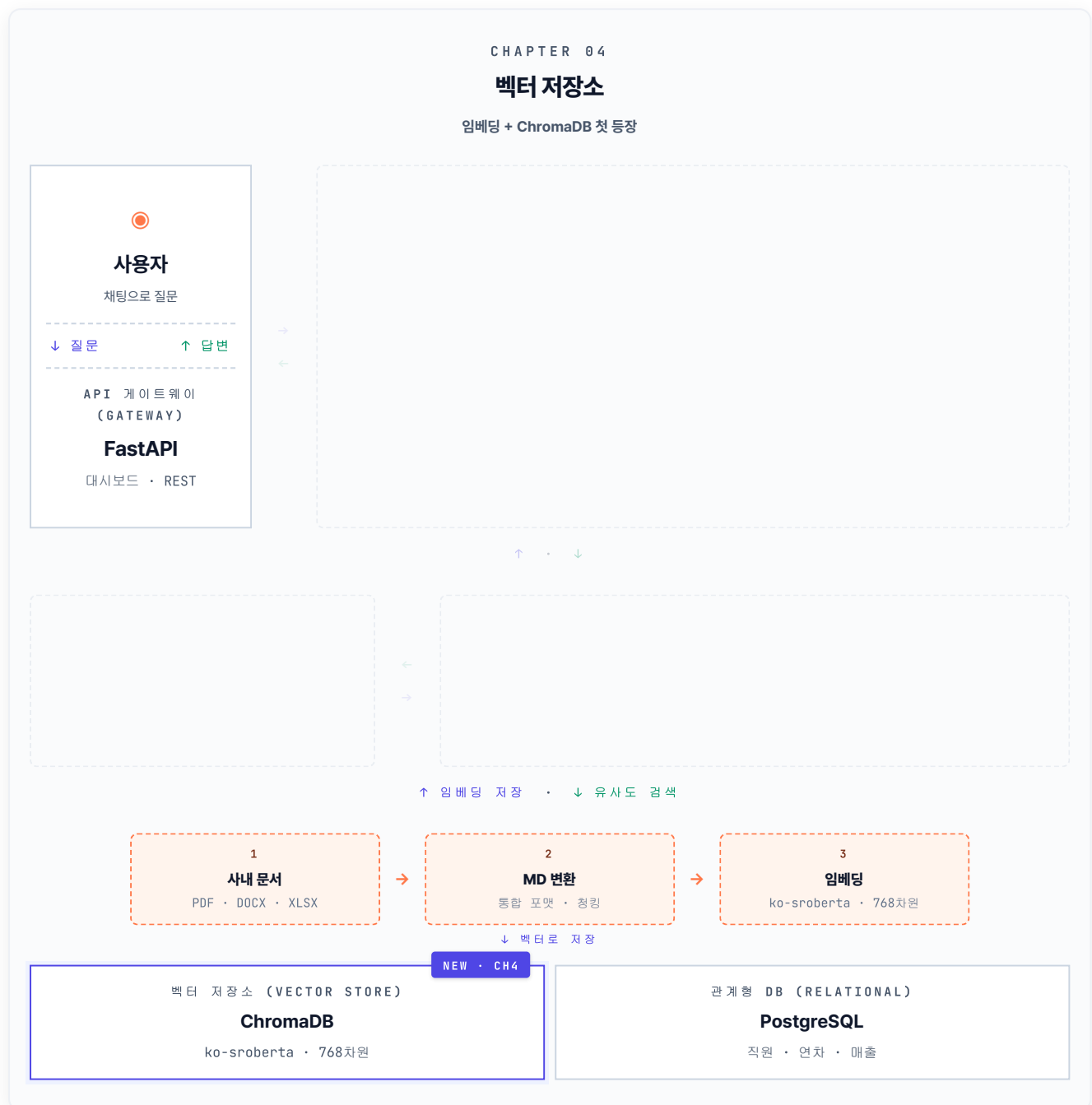
4.7 전체 구성도에서 챗터 4의 자리

챗터 4는 챗터 3에서 정한 **문서 규칙 네 장**(포맷·청킹·메타데이터·재인덱싱)을 실제 코드로 옮겼습니다.

사내 문서를 MD로 손질하고, 500자 단위로 다지고, 768차원 벡터로 양념해 ChromaDB라는 냉장고에 채웠습니다. 챗터 2의 사용자·FastAPI·PostgreSQL 위에 **벡터 저장소**가 처음으로 얹힌 순간입니다.

아래 카드는 이번 챗터에서 새로 자리잡은 부품을 한 장으로 묶은 룩북입니다. 박스 가운데 아래쪽에 등장한 **사내 문서 → MD 변환 → 임베딩 → ChromaDB** 흐름이 챗터 4의 NEW입니다. 챗터 5부터는 이 ChromaDB에 사용자 질문이 도달하면서 RAG 체인이 본격적으로 작동하기 시작합니다.

CH 04 · 문서를 지식으로 - ChromaDB NEW



이전 : 사용자 · FastAPI · PostgreSQL · 새로 : 사내 문서 → MD → 임베딩 → ChromaDB

그림 4-9. 챗터 4의 결과. 사내 문서가 임베딩을 거쳐 ChromaDB에 새로 자리를 잡았습니다

용어 정리

본문 속 표현	진짜 용어	정식 정의
"손질"	파싱 (Parsing)	PDF·DOCX·XLSX에서 순수 텍스트를 추출하는 과정
"다지기"	청킹 (Chunking)	긴 텍스트를 벡터 DB에 적합한 크기로 분할. 고정 500자 + 오버랩 100자
"양념"	임베딩 (Embedding)	텍스트 의미를 768차원 숫자 벡터로 변환
"냉장고"	벡터 DB (Vector Database)	벡터를 저장하고 유사한 벡터를 빠르게 검색하는 특수 DB
"의미 유사도"	코사인 유사도 (Cosine Similarity)	두 벡터 사이 각도로 유사도를 측정. 1에 가까울수록 유사
"덧어쓰기"	업서트 (Upsert)	같은 ID가 있으면 업데이트, 없으면 삽입하는 연산

검색이 돌기 시작하자 옆자리 동료가 화면을 들여다봤습니다.

동료 : "이제 단어가 달라도 찾아오네요. '휴가 규정'이라고 했는데 '연차 유급휴가'가 잡히는 거 신기하네요."

오픈이가 1위 점수 0.71을 가리켰습니다.

오픈이 : "1위가 71%까지 올라왔으니 일단 도착선은 넘었네요. 그런데 1·2·3위 사이 격차가 0.15밖에 안 돼서 누가 진짜 답인지 좀 애매하긴 해요."

동료 : "그건 어떻게 해결해요?"

오픈이 : "그게 챗터 8 검색 품질 튜닝에서 다룰 일이에요. 일단 챗터 5에서는 이 검색 결과를 LLM에 붙여서 자연어 답변부터 받아 봐요."

이것만은 기억하자

- **문서 → 지식은 네 단계를 거칩니다.** 손질(파싱), 다지기(청킹), 양념(임베딩), 냉장고(ChromaDB 저장). 주방의 4단계가 그대로 코드의 4단계입니다.
- **파싱 결과를 먼저 눈으로 확인하세요.** 챕터 3의 **Garbage In, Garbage Out** 원칙이 여기서 적용됩니다. 텍스트가 깨져 있으면 뒤에서 아무리 고쳐도 소용없습니다. `data/markdown/`의 결과를 반드시 검증합니다.
- **벡터 검색은 키워드 매칭이 아닙니다.** "휴가 규정"이라고 물어도 "연차 유급휴가"가 잡힙니다. 의미가 가까우면 찾아냅니다. 다음 챕터 5에서는 이 검색 결과를 **LCEL 체인**에 붙여 실제 답변을 만들어 냅니다.